Student Course Management System

Individual Project Report

Student Name	Altarek Mohamed Alsaied Tawfiek
Student ID	11188
Course	Fundamentals C++
Instructor	Dr. Mostafa Ashrif Mohamed
Submission	March 26, 2026

Submitted to:

Egyptian Military Academy

Department of Computer Science

1. Introduction

This project presents a console-based **Student Course Management System** developed using the C++ programming language. The primary objective of this system is to facilitate the management of student records and their corresponding course enrollments in an organized and efficient manner.

The application enables the user to perform essential operations including: adding and removing students, searching by unique ID, displaying all records, enrolling students in courses, and sorting the student list by GPA. These capabilities make the system a practical tool for academic data management.

The project is built upon core Object-Oriented Programming (OOP) principles such as **inheritance**, **encapsulation**, and the use of the **C++ Standard Template Library (STL)** — specifically the *vector* container and the *algorithm* library for sorting operations. This ensures that the code is modular, maintainable, and extensible.

GitHub Repository: https://github.com/Tarekattallah/Student-Course-Management-System

Table of Contents

1.	Introduction	2
2.	Table of Contents	3
3.	Detailed Project Description	4
4.	Project Beneficiaries	4
5.	Detailed Analysis	5
6.	Task Breakdown Structure	5
7.	Time Plan (Gantt Chart)	6
8.	System Architecture and Design (UML)	7
9.	Testing Scenarios and Results	8
10.	End-User Guide	9
11.	Conclusion	9
12.	References	10

3. Detailed Project Description

The Student Course Management System is a fully interactive console application that provides a structured menu-driven interface. Users interact with the program through a numbered menu that maps to specific system operations.

Main Menu Functions

Option	Feature	Description

Stakeholder	Benefit
University Administration	Enables quick management of student records, reducing administrative workload and minimizing human error.

1	**Add Student**	Prompts for name, unique ID, and GPA (0.0–4.0). Validates all inputs before storing.
2	**Remove Student**	Removes a student record permanently by entering their ID.
3	**Search Student**	Retrieves and displays full student information for a given ID.
4	**Display All Students**	Lists all stored students with their ID, name, GPA, and enrolled courses.
5	**Enroll in Course**	Adds a course to a student's schedule. Duplicate enrollment is automatically prevented.
6	**Show Courses**	Displays all courses currently enrolled for a specified student.
7	**Sort by GPA**	Sorts and displays the entire student list in descending order of GPA.
8	**Exit**	Safely terminates the program.

The program is entirely memory-resident; all data is stored in a std::vector<Student> and is not persisted between sessions. The architecture follows a clean separation between the *Person* base class and the *Student* derived class.

4. Project Beneficiaries

Students	Provides clear visibility into personal academic data including GPA and enrolled courses.
Instructors & Advisors	Facilitates easy access to student performance data for academic advising purposes.
Developers & Learners	Serves as a practical learning project for practicing OOP, STL, and software design in C++.

5. Detailed Analysis

The system handles student data with emphasis on **data integrity** and **input validation**. The following constraints are enforced at runtime:

Field / Operation	Validation Rule
Student ID	Must be a unique positive integer. Duplicate IDs are rejected with an error message.
Student Name	Must be a non-empty string. Whitespace-only names are not accepted.
GPA	Must be a floating-point number in the range [0.0, 4.0]. Values outside this range trigger an error.
Course Name	Must be a non-empty string. A student cannot be enrolled in the same course twice.
Search/Remove	If the specified student ID does not exist, a descriptive error message is displayed.

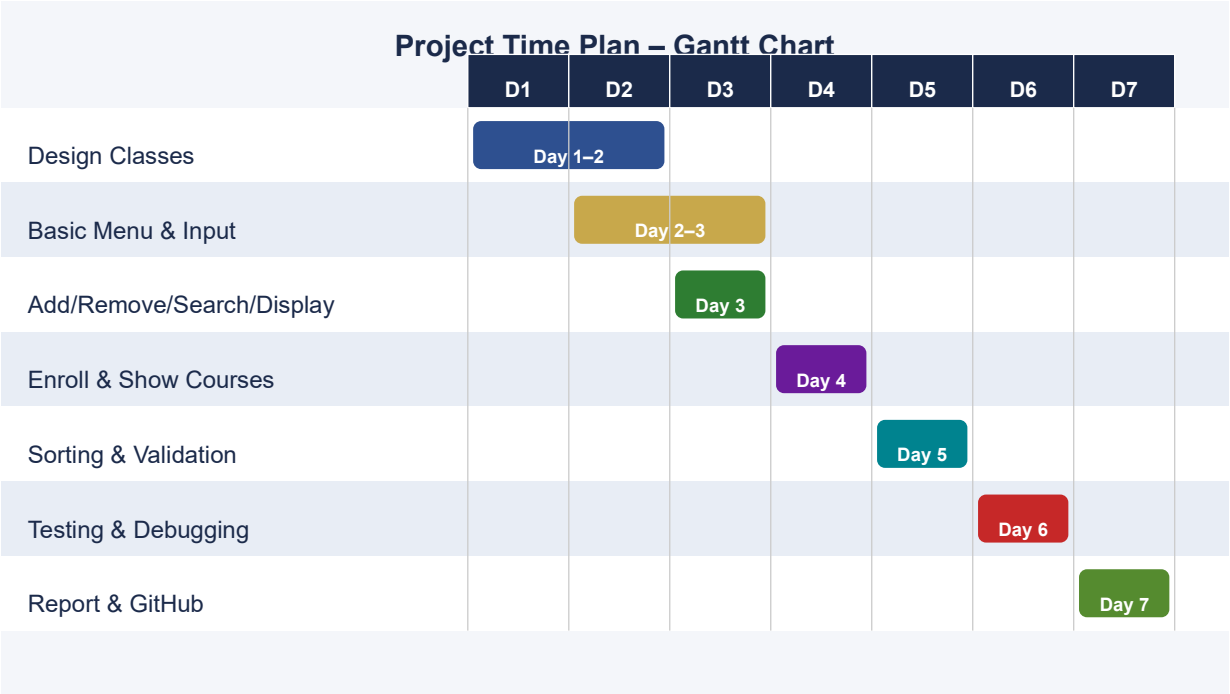
The design ensures that the system remains robust against invalid inputs while providing meaningful feedback to the user at all times. The STL std::sort with a custom comparator lambda is used to implement efficient GPA-based sorting.

6. Task Breakdown Structure

Task 1	Design class hierarchy (Person base class and Student derived class)
Task 2	Implement the main menu loop and basic input/output handling
Task 3	Develop Add, Remove, Search, and Display Student functions
Task 4	Implement Course Enrollment and Show Courses functionality
Task 5	Add GPA-based sorting using STL std::sort and lambda comparator
Task 6	Integrate input validation: unique ID check, GPA range, duplicate course prevention
Task 7	Conduct functional testing and fix identified bugs
Task 8	Write project report and set up GitHub repository

7. Time Plan (Gantt Chart)

The project was designed to be completed within a single week. The following Gantt chart illustrates the distribution of tasks across the seven working days:



Period	Work Completed
Day 1–2	Class design: Person and Student hierarchy, attribute definitions, method signatures
Day 3	Implemented Add, Remove, Search, and Display All Students functions
Day 4	Implemented Enroll in Course and Show Student Courses functionality
Day 5	Added GPA-based sorting via STL and integrated all input validation rules
Day 6	Full system testing, edge-case handling, and bug resolution
Day 7	Prepared final report, formatted documentation, pushed code to GitHub

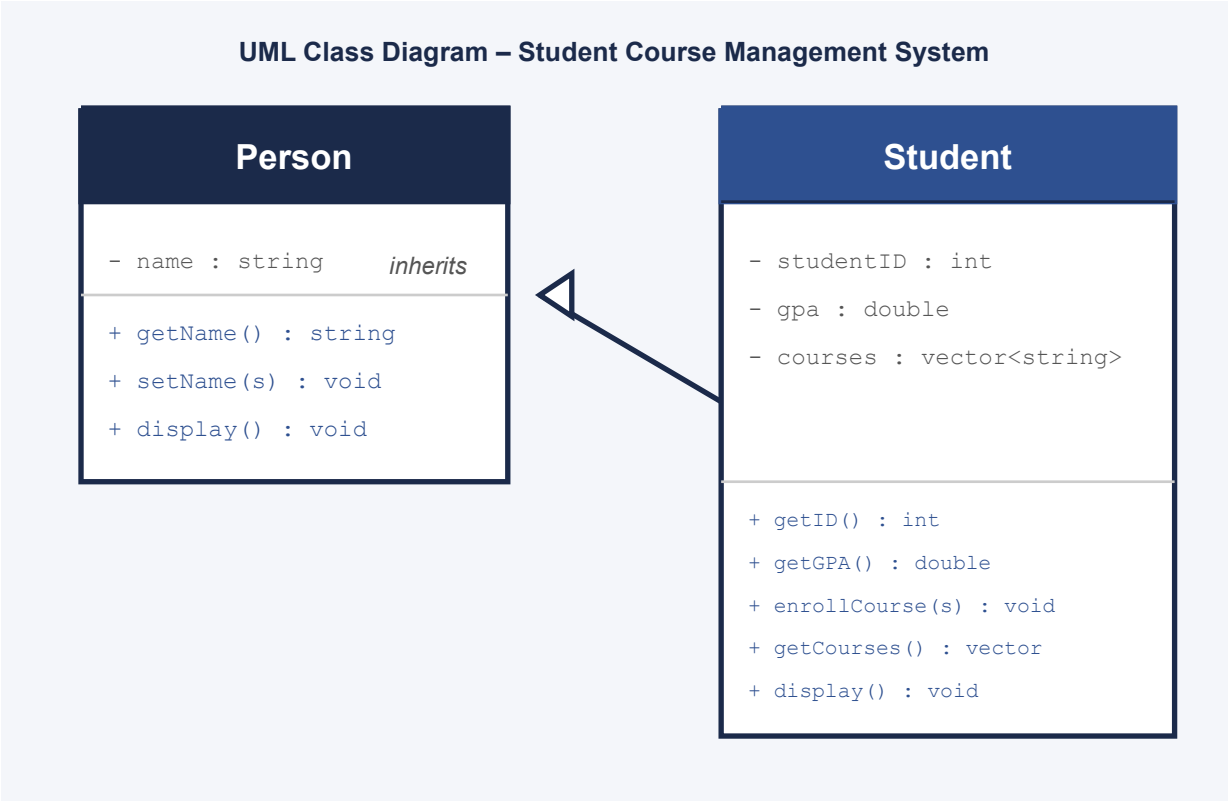
8. System Architecture and Design

8.1 Class Hierarchy Overview

The system follows a clear object-oriented hierarchy. The **Person** class serves as the base class encapsulating the common attribute *name*. The **Student** class publicly inherits

from Person and extends it with academic-specific attributes: *studentID*, *gpa*, and a vector<string> for course enrollment.

8.2 UML Class Diagram



8.3 Key OOP Concepts Applied

Inheritance	Student publicly inherits from Person, reusing name management and the virtual display() method.
Encapsulation	All class members are declared private. Access is controlled exclusively through public getter and setter methods.
Abstraction	The menu system hides the internal complexity of data management from the end user.

STL Usage	std::vector stores all student objects dynamically. std::sort with a lambda comparator handles GPA-based sorting.

9. Testing Scenarios and Results

A comprehensive set of test cases was executed to verify the correctness and robustness of the system. All critical paths and edge cases were covered.

ID	Test Case	Input	Expected Output	Result
TC-01	Add student – valid data	`Name: Ahmed, ID: 101, GPA: 3.5`	Student added successfully	**PASS**
TC-02	Add student – duplicate ID	`ID: 101 (already exists)`	Error: ID already exists	**PASS**
TC-03	Add student – GPA out of range	`GPA = 5.0`	Error: GPA must be between 0.0 and 4.0	**PASS**
TC-04	Add student – GPA boundary (min)	`GPA = 0.0`	Student added successfully	**PASS**
TC-05	Add student – GPA boundary (max)	`GPA = 4.0`	Student added successfully	**PASS**
TC-06	Enroll student in course	`ID: 101, Course: Math`	Course enrolled successfully	**PASS**
TC-07	Enroll – duplicate course	`ID: 101, Course: Math (again)`	Error: Already enrolled in this course	**PASS**
TC-08	Search – existing student	`ID: 101`	Student info displayed correctly	**PASS**
TC-09	Search – non-existent student	`ID: 999`	Error: Student not found	**PASS**
TC-10	Remove student	`ID: 101`	Student removed successfully	**PASS**
TC-11	Sort by GPA – multiple students	`Add 3 students with GPAs: 2.1, 3.8, 1.5`	List displayed in order: 3.8, 2.1, 1.5	**PASS**
TC-12	Display all – empty list	`No students added`	Message: No students found	**PASS**

Note: Screenshots of the running program for each menu option should be captured during the live demonstration and can be appended as supplementary material.

Testing link: https://drive.google.com/file/d/1FQsTE-pioWKxVbvSOlpLaan1Olj7YEze/view?usp=drive_link

10. End-User Guide

10.1 Prerequisites

- Microsoft Visual Studio (2019 or later) or any C++17-compatible IDE
- Windows 10 / 11 operating system
- No additional libraries required — the system uses only standard C++ headers

10.2 How to Build and Run

Step 1: Open the project folder in Visual Studio.

Step 2: Open the solution file StudentSystem.sln.

Step 3: Build the solution using **Ctrl + Shift + B**. Ensure zero errors.

Step 4: Run the program using **Ctrl + F5** (Run Without Debugging).

Step 5: The main menu will appear in the console window.

Step 6: Enter the number corresponding to your desired operation and follow the prompts.

10.3 Sample Usage Walkthrough

To add a new student:

```
Select option [1] from the main menu.
Enter student name: Ahmed Ali
Enter student ID: 201
Enter GPA (0.0 - 4.0): 3.75
→ Output: "Student added successfully."
```

To enroll the student in a course:

```
Select option [5] from the main menu.
Enter student ID: 201
Enter course name: Data Structures
→ Output: "Course enrolled successfully."
```

11. Conclusion

The **Student Course Management System** successfully demonstrates the practical application of Object-Oriented Programming concepts in C++. Through the implementation of class inheritance, encapsulation, and STL data structures, the project achieves a clean, modular architecture that handles all required operations efficiently.
The development process reinforced key software engineering practices including input validation, separation of concerns, and systematic testing. All twelve test cases passed successfully, confirming the system's correctness and robustness.

Potential future enhancements could include: persisting student data to a file using file I/O streams, implementing a graphical user interface (GUI) using Qt or wxWidgets, adding a database backend for scalability, and introducing a login/authentication system for role-based access. However, for the current assignment scope, the console-based implementation fully satisfies all stated requirements.

12. References

[1] cppreference.com – C++ Standard Library Reference. https://en.cppreference.com

[2] Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.

[3] Lippman, S., Lajoie, J., & Moo, B. (2012). C++ Primer (5th ed.). Addison-Wesley.

[4] University Course Lectures – Object-Oriented Programming with C++. Egyptian MilitaryAcademy.

[5] Microsoft Visual Studio Documentation. https://docs.microsoft.com/en-us/visualstudio/

[6] GitHub Repository – Student Course Management System.
https://github.com/Tarekattallah/Student-Course-Management-System